

Laboratorio incremental de Kubernetes (versión completa e adaptada a VM con NAT/SSH)

Obxectivo: partir dun **Hello World** funcional e evolucionar cara a un despregue web completo engadindo **Services**, **Volumes**, **ConfigMaps/Secrets**, **Probes** e **Ingress**, todo en Minikube.

Cada etapa inclúe explicacións, código e instrucións para acceso tanto dende a VM como dende o host en contornos con rede NAT e acceso por SSH.

0) Requisitos e preparación do entorno

1. **Minikube + kubectl** instalados (ver apuntamentos).
2. Arrincar Minikube empregando Docker como driver (ou o que uses normalmente):

```
minikube delete || true  
minikube start --driver=docker
```

3. (Recomendado) Activar **addons** que imos usar máis tarde:

```
minikube addons enable metrics-server  
minikube addons enable ingress
```

4. Namespace de traballo:

```
kubectl create namespace lab-k8s || true  
kubectl config set-context --current --namespace=lab-k8s
```

Podes volver ao *namespace* por defecto con `kubectl config set-context --current --namespace=default`.

1) Hello World imperativo (Deployment + Service NodePort)

Nesta primeira etapa lanzamos un **echoserver** que devolve información da petición. Facémolo con comandos *imperativos* para ver resultados rápidos.

1.1 Crear o Deployment

```
kubectl create deployment hello-world --image=registry.k8s.io/echoserver:1.10 --port=8080
```

Que fai?

Crea un **Deployment** chamado `hello-world` que xestiona Pods co contedor `echoserver` escoitando en `8080`.

Se o Pod entra en `CrashLoopBackOff`, comproba logs con:

```
kubectl logs -l app=hello-world --tail=50
```

1.2 Expor o Service tipo NodePort

```
kubectl expose deployment hello-world --type=NodePort --port=8080 --target-port=8080
```

Que fai?

Crea un **Service** que publica o Deployment vía **NodePort** (porto externo aleatorio entre 30000-32767).

1.3 Probar acceso

Opción A: entorno con GUI

```
minikube service hello-world
```

Amosarase unha táboa na que se amosa a URL a través da cal se accede ao servizo. Podes empregar CURL para probar o funcionamento:

```
curl http://<ip>/<porto>
```

Opción B: acceso dende o host (port-forward interno)

1. Executa o seguinte comando (na VM):

```
kubectl port-forward --address 0.0.0.0 service/hello-world 8080:8080
```

2. Asegúrate de que o `port-forwarding` estea correctamente configurado en `VirtualBox` (ou usa SSH cun túnel).

3. No navegador do host abre:

<http://localhost:8080>

Deberías ver unha resposta textual con `Hostname`, `Request headers`, etc.

Limpar (opcional)

```
kubectl delete svc hello-world
kubectl delete deploy hello-world
```

2) Despreque web declarativo (Nginx + Service ClusterIP)

Pasamos a YAML declarativos. Lanzaremos Nginx e o exporemos internamente (ClusterIP).

Ficheiro: 02-web-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Aplicar e verificar

```
kubectl apply -f 02-web-deployment.yaml
kubectl get deploy,po,svc -l app=web
```

Acceso dende a VM e o host

1 Na VM:

```
kubectl port-forward --address 0.0.0.0 service/web 8080:80
```

2 No host (se estás por SSH):

```
ssh -L 8080:localhost:8080 usuario@IP_DA_VM
```

3 Probar:

```
http://localhost:8080
```

Limpeza dos recursos

Borramos os obxectos creados a partir do ficheiro YAML:

```
kubectl delete -f 02-web-deployment.yaml
```

3) Engadir contido co ConfigMap montado como Volume

Serviremos unha páxina personalizada en Nginx dende un ConfigMap.

Ficheiro: 03-web-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-content
data:
  index.html: |
    <!doctype html>
    <html>
      <head><meta charset="utf-8"><title>K8s Lab</title></head>
      <body>
        <h1>Benvido ao Laboratorio K8s</h1>
        <p>Servido dende un <code>ConfigMap</code>.</p>
      </body>
    </html>
```

Ficheiro: 03-web-deployment-cm.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          volumeMounts:
            - name: web-root
              mountPath: /usr/share/nginx/html
      volumes:
        - name: web-root
          configMap:
            name: web-content
```

Ficheiro: 03-web-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Aplicar e probar

```
kubectl apply -f 03-web-configmap.yaml
kubectl apply -f 03-web-deployment-cm.yaml
```

```
kubectl apply -f 03-web-svc.yaml
kubectl rollout status deploy/web
```

Acceso

```
kubectl port-forward --address 0.0.0.0 svc/web 8080:80
```

No navegador do host accedemos a <http://localhost:8080>

Limpeza

```
kubectl delete -f 03-web-svc.yaml
kubectl delete -f 03-web-deployment-cm.yaml
kubectl delete -f 03-web-configmap.yaml
```

4) Persistencia con PVC

Agora en vez de **ConfigMap** usaremos un volume persistente.

Ficheiro: [04-web-pvc.yaml](#)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Ficheiro: [04-web-deployment-pvc.yaml](#)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
```

```

metadata:
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.25-alpine
      ports:
        - containerPort: 80
      volumeMounts:
        - name: web-data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: web-data
      persistentVolumeClaim:
        claimName: web-pvc

```

Ficheiro: `04-writer-pod.yaml` (Pod auxiliar que escribe un `index.html` no PVC):

```

apiVersion: v1
kind: Pod
metadata:
  name: writer
spec:
  restartPolicy: Never
  containers:
    - name: writer
      image: alpine:3.20
      command: ["/bin/sh", "-c"]
      args:
        - |
          echo '<!doctype html><html><head><meta charset="utf-8">
          <title>PVC</title></head>
          <body><h1>Contido persistente en PVC</h1><p>Este index.html persiste
          aínda que os Pods se borren.</p></body></html>' > /data/index.html
          ; sleep 2
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: web-pvc

```

Ficheiro: `04-web-svc.yaml`

```

apiVersion: v1
kind: Service
metadata:

```

```
name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Aplicamos

```
kubectl apply -f 04-web-pvc.yaml
kubectl apply -f 04-web-deployment-pvc.yaml
kubectl apply -f 04-writer-pod.yaml
kubectl apply -f 04-web-svc.yaml
kubectl rollout status deploy/web
kubectl get pods -l app=web
```

Acceso:

```
kubectl port-forward --address 0.0.0.0 svc/web 8080:80
```

Limpeza

```
kubectl delete -f 04-web-svc.yaml
kubectl delete -f 04-writer-pod.yaml
kubectl delete -f 04-web-deployment-pvc.yaml
kubectl delete -f 04-web-pvc.yaml
```

5) Probes: liveness, readiness e startup (saúde da app)

Obxectivo: engadir verificacións que determinen se a app está viva (**liveness**), lista para recibir tráfico (**readiness**) e lista por primeira vez (**startup**).

Por que importa?

- *Readiness* controla se o **Service** envía tráfico ao Pod.
- *Liveness* permite **reiniciar** o contedor se está “zombie”.
- *Startup* evita falsos positivos mentres a app arrinca lento.

Ficheiro: 05-web-probes.yaml


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 2
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3
          livenessProbe:
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          startupProbe:
            httpGet: { path: /, port: 80 }
            failureThreshold: 30
            periodSeconds: 2
```

Ficheiro: 05-web-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Aplicar e observar

```
kubectl apply -f 05-web-probes.yaml
kubectl apply -f 05-web-svc.yaml
kubectl rollout status deploy/web
kubectl describe pod -l app=web | grep -i probe -n || true
```

Proba de comportamento (simular fallo de liveness):

1. Entra nun Pod:

```
kubectl exec -it deploy/web -- sh
```

2. Rompe o Nginx no contedor (por exemplo, elimina o proceso):

```
kill 1
exit
```

3. Observa como **Kubelet reinicia o contedor** segundo a liveness probe:

```
kubectl get pods -l app=web -w
```

Acceso (túnel)

```
kubectl port-forward --address 0.0.0.0 svc/web 8080:80
curl -I http://localhost:8080
```

Limpeza

```
kubectl delete -f 05-web-probes.yaml
```

6) Publicación con Ingress (Minikube)

Obxectivo: publicar a app por HTTP baixo un **nome de host** usando **Ingress** (co add-on de Minikube).

Xa activamos `minikube addons enable ingress` ao comezo.

Ficheiro: `06-web-with-svc.yaml` (Service + Deployment básico)

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: web
spec:
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports: [ { containerPort: 80 } ]
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP

```

Ficheiro: 06-web-ingress.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: lab.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80

```

Aplicar

```

kubectl apply -f 06-web-with-svc.yaml
kubectl apply -f 06-web-ingress.yaml
kubectl get ingress

```

Acceso segundo entorno

- **Opción A (VM con GUI):**

```
minikube ip
```

Engade no `/etc/hosts` (na VM):

`MINIKUBE_IP lab.local`

Proba en navegador da VM: `http://lab.local`

- **Opción B (host por SSH NAT):**

1. Obtén a IP de Minikube na **VM**:

```
minikube ip
```

2. Dende o host coa configuración actual de rede de virtualbox é imposible, así que ímolo comprobar dende a liña de comandos da VM:

```
curl -H "Host: lab.local" http://192.168.49.2
```

Limpeza

```
kubectl delete -f 06-web-ingress.yaml  
kubectl delete -f 06-web-with-svc.yaml
```

7) ConfigMaps e Secrets (env e volumes + boas prácticas)

Obxectivo: separar configuración e credenciais da imaxe; inxectar parámetros por **entorno** e **ficheiros**.

Ficheiro: `07-config.yaml`

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: app-config  
data:  
  APP_MODE: "production"  
  APP_FEATURE_X: "enabled"  
---  
apiVersion: v1
```

```
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_USER: YWRtaW4= # admin
  DB_PASS: c2VjcmV0 # secret
```

Ficheiro: 07-web-env-and-files.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          envFrom:
            - configMapRef: { name: app-config }
            - secretRef: { name: app-secret }
          volumeMounts:
            - name: confd
              mountPath: /etc/webconfig
      volumes:
        - name: confd
          projected:
            sources:
              - configMap:
                  name: app-config
              - secret:
                  name: app-secret
```

Aplicar e verificar

```
kubectl apply -f 07-config.yaml
kubectl apply -f 07-web-env-and-files.yaml
kubectl exec -it deploy/web -- sh -c 'printenv | egrep "^APP_|^DB_" ; ls -l /etc/webconfig'
```

Acceso

```
kubectl port-forward --address 0.0.0.0 deploy/web 8080:80
curl -I http://localhost:8080
```

Boas prácticas rápidas

- Nunca gardes contrasinais en claro en YAML (usa **Secret** ou ferramentas tipo SealedSecrets/SOPS).
- Versiona ConfigMaps/Secrets con **nombres versionados** se despregas cambios que deben gatillar *rollout*.
- Preferir **envFrom** para variables e **volumes proxectados** para expoñer múltiples fontes a ficheiros.

Limpeza

```
kubectl delete -f 07-web-env-and-files.yaml
kubectl delete -f 07-config.yaml
```

8) Escalado e actualizacións (HPA + Rolling updates + rollback)

8.1 Escalado horizontal (HPA)

Requisitos: **metrics-server** habilitado (fixemos no paso 0).

Ficheiro: **08-web-hpa.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - name: nginx
          image: nginx:1.25-alpine
          ports: [ { containerPort: 80 } ]
          resources:
            requests: { cpu: "50m", memory: "64Mi" }
            limits: { cpu: "200m", memory: "128Mi" }
---
apiVersion: v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
```

```
  apiVersion: apps/v1
  kind: Deployment
  name: web
  minReplicas: 2
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

Aplicar e observar

```
kubectl apply -f 08-web-hpa.yaml
kubectl get hpa,pods -w
```

Xerar carga (pod de carga simple)

```
kubectl run load --image=busybox --restart=Never -- /bin/sh -c "while true; do
wget -q -O- http://web; done"
# Observa como medra o número de réplicas ao pasar do 60% de uso CPU (pode tardar
1-2 min)
```

Limpeza da carga

```
kubectl delete pod load
```

8.2 Actualizacións (rolling update) e rollback

Actualizar imaxe

```
kubectl set image deploy/web nginx=nginx:1.27-alpine
kubectl rollout status deploy/web
kubectl rollout history deploy/web
```

Rollback se algo vai mal

```
kubectl rollout undo deploy/web
```

Estratexia explícita (opcional)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
```

Pausar / reanudar despregue (para canary manual)

```
kubectl rollout pause deploy/web
# editar pods template (labels/selectors/recursos...)
kubectl rollout resume deploy/web
```

9) Limpieza final

```
kubectl delete ns lab-k8s --wait=false
kubectl config set-context --current --namespace=default
```

💡 Notas engadidas

- Incluído método de **acceso dende o host** en cada práctica.
- Evitada confusión entre `localhost` da VM e do host.
- Corrixidos exemplos con `CrashLoopBackOff`.
- Todos os port-forward abren con `--address 0.0.0.0` por defecto.